

Closira Backend — Complete Development Roadmap

PHASE 0: PROJECT SETUP

0.1 Prerequisites

Make sure you have these installed:

- Python 3.10+
- pip
- git

0.2 Create the repo and folder structure

bash

```
mkdir closira && cd closira
```

```
git init
```

```
mkdir -p backend/app/routes backend/app/tasks backend/tests
```

```
cd backend
```

```
touch app/__init__.py app/main.py app/models.py app/schemas.py app/database.py
```

```
app/logger.py
```

```
touch app/routes/__init__.py app/routes/enquiry.py app/routes/health.py
```

```
touch app/tasks/__init__.py app/tasks/sop_matcher.py
```

```
touch tests/__init__.py tests/test_enquiry.py
```

```
touch requirements.txt .env .gitignore README.md
```

Your final folder structure should look like this:

```
backend/
├── app/
│   ├── __init__.py
│   ├── main.py
│   ├── models.py
│   ├── schemas.py
│   ├── database.py
│   ├── logger.py
│   └── routes/
│       ├── __init__.py
│       ├── enquiry.py
│       └── health.py
│   └── tasks/
│       ├── __init__.py
│       └── sop_matcher.py
├── tests/
│   ├── __init__.py
│   └── test_enquiry.py
```

```
|— requirements.txt
|— .env
|— .gitignore
|— README.md
```

0.3 requirements.txt — paste this exactly

```
fastapi==0.111.0
uvicorn[standard]==0.29.0
sqlalchemy==2.0.30
pydantic==2.7.1
pydantic-settings==2.2.1
python-dotenv==1.0.1
pytest==8.2.0
httpx==0.27.0
pytest-asyncio==0.23.6
```

0.4 Install dependencies

```
bash
cd backend
python -m venv venv
source venv/bin/activate    # Windows: venv\Scripts\activate
pip install -r requirements.txt
```

0.5 .gitignore

```
venv/
__pycache__/
*.pyc
.env
*.db
.DS_Store
```

0.6 .env

```
DATABASE_URL=sqlite:///./closira.db
```

PHASE 1: DATABASE LAYER

1.1 app/database.py — full file

This sets up SQLAlchemy with SQLite. Every table in your app will be created from here.

```
python
from sqlalchemy import create_engine
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker
from dotenv import load_dotenv
import os
```

```

load_dotenv()

DATABASE_URL = os.getenv("DATABASE_URL", "sqlite:///./closira.db")

# connect_args only needed for SQLite (allows multi-thread access)
engine = create_engine(
    DATABASE_URL,
    connect_args={"check_same_thread": False}
)

SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

Base = declarative_base()

def get_db():
    """Dependency that provides a DB session and closes it after the request."""
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

```

1.2 app/models.py — full file

Four tables: enquiries, messages, status_timeline, followups.

```

python
from sqlalchemy import Column, String, Integer, Text, DateTime, ForeignKey, Boolean
from sqlalchemy.orm import relationship
from datetime import datetime
import uuid
from app.database import Base

def generate_uuid():
    return str(uuid.uuid4())

class Enquiry(Base):
    __tablename__ = "enquiries"

    id = Column(String, primary_key=True, default=generate_uuid)
    customer_name = Column(String, nullable=False)
    channel = Column(String, nullable=False)    # whatsapp | email | call
    message = Column(Text, nullable=False)
    status = Column(String, default="new")      # new | processing | sop_matched |
escalated
    sop_match = Column(String, nullable=True)   # name of matched SOP
    suggested_response = Column(Text, nullable=True)
    escalation_reason = Column(Text, nullable=True)

```

```
created_at = Column(DateTime, default=datetime.utcnow)
updated_at = Column(DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)

messages = relationship("Message", back_populates="enquiry", cascade="all, delete")
timeline = relationship("StatusTimeline", back_populates="enquiry", cascade="all, delete")
followups = relationship("FollowUp", back_populates="enquiry", cascade="all, delete")
```

```
class Message(Base):
    __tablename__ = "messages"

    id = Column(String, primary_key=True, default=generate_uuid)
    enquiry_id = Column(String, ForeignKey("enquiries.id"), nullable=False)
    sender = Column(String, nullable=False)      # customer | agent | ai
    content = Column(Text, nullable=False)
    timestamp = Column(DateTime, default=datetime.utcnow)

    enquiry = relationship("Enquiry", back_populates="messages")
```

```
class StatusTimeline(Base):
    __tablename__ = "status_timeline"

    id = Column(String, primary_key=True, default=generate_uuid)
    enquiry_id = Column(String, ForeignKey("enquiries.id"), nullable=False)
    status = Column(String, nullable=False)
    note = Column(Text, nullable=True)
    timestamp = Column(DateTime, default=datetime.utcnow)

    enquiry = relationship("Enquiry", back_populates="timeline")
```

```
class FollowUp(Base):
    __tablename__ = "followups"

    id = Column(String, primary_key=True, default=generate_uuid)
    enquiry_id = Column(String, ForeignKey("enquiries.id"), nullable=False)
    scheduled_at = Column(DateTime, nullable=False)
    message_template = Column(Text, nullable=True)
    done = Column(Boolean, default=False)
    created_at = Column(DateTime, default=datetime.utcnow)

    enquiry = relationship("Enquiry", back_populates="followups")
```

PHASE 2: SCHEMAS (Pydantic)

2.1 app/schemas.py — full file

Schemas define what your API accepts and returns. Keep input schemas (request bodies) and output schemas (responses) separate.

```
python
from pydantic import BaseModel, Field
from typing import Optional, List
from datetime import datetime
from enum import Enum
```

— Enums

```
class ChannelEnum(str, Enum):
    whatsapp = "whatsapp"
    email = "email"
    call = "call"
```

```
class StatusEnum(str, Enum):
    new = "new"
    processing = "processing"
    sop_matched = "sop_matched"
    escalated = "escalated"
```

```
class SenderEnum(str, Enum):
    customer = "customer"
    agent = "agent"
    ai = "ai"
```

— Enquiry

```
class EnquiryCreate(BaseModel):
    customer_name: str = Field(..., example="Sarah M.", description="Full name of the customer")
    channel: ChannelEnum = Field(..., example="whatsapp", description="Inbound channel")
    message: str = Field(..., example="Hi, I'd like to book an appointment for next week.", description="Customer's message")
```

```
class EnquiryCreatedResponse(BaseModel):
    job_id: str = Field(..., description="The enquiry ID to track processing status")
    status: str = Field(default="processing")
    message: str = Field(default="Enquiry received. Processing in background.")
```

```
class Config:
    from_attributes = True
```

— Follow-up

```
class FollowUpCreate(BaseModel):
    delay_minutes: int = Field(..., ge=1, example=30, description="Minutes from now to
schedule the follow-up")
    message_template: Optional[str] = Field(None, example="Hi {name}, just following up on
your enquiry!")

class FollowUpResponse(BaseModel):
    id: str
    enquiry_id: str
    scheduled_at: datetime
    message_template: Optional[str]
    done: bool

class Config:
    from_attributes = True
```

— Escalation

```
class EscalateCreate(BaseModel):
    reason: str = Field(..., example="Customer is very unhappy and requesting a manager.",
description="Reason for escalation")

class EscalateResponse(BaseModel):
    id: str
    status: str
    escalation_reason: str

class Config:
    from_attributes = True
```

— History

```
class MessageOut(BaseModel):
    id: str
    sender: str
    content: str
    timestamp: datetime

class Config:
    from_attributes = True
```

```

class TimelineOut(BaseModel):
    status: str
    note: Optional[str]
    timestamp: datetime

    class Config:
        from_attributes = True

class EnquiryHistoryResponse(BaseModel):
    id: str
    customer_name: str
    channel: str
    status: str
    sop_match: Optional[str]
    suggested_response: Optional[str]
    escalation_reason: Optional[str]
    created_at: datetime
    messages: List[MessageOut]
    timeline: List[TimelineOut]

    class Config:
        from_attributes = True

```

— Health

```

class HealthResponse(BaseModel):
    status: str
    database: str

```

PHASE 3: STRUCTURED LOGGER

3.1 app/logger.py — full file

JSON structured logging. Every key event gets logged with a consistent format. This is what makes your submission look professional.

```

python
import logging
import json
from datetime import datetime

class JSONFormatter(logging.Formatter):
    def format(self, record):
        log_obj = {

```

```

        "timestamp": datetime.utcnow().isoformat(),
        "level": record.levelname,
        "event": record.getMessage(),
        "logger": record.name,
    }
    # Attach any extra fields passed via extra={}
    for key, value in record.__dict__.items():
        if key not in (
            "args", "asctime", "created", "exc_info", "exc_text",
            "filename", "funcName", "id", "levelname", "levelno",
            "lineno", "module", "msecs", "message", "msg", "name",
            "pathname", "process", "processName", "relativeCreated",
            "stack_info", "thread", "threadName"
        ):
            log_obj[key] = value
    return json.dumps(log_obj)

def get_logger(name: str) -> logging.Logger:
    logger = logging.getLogger(name)
    if not logger.handlers:
        handler = logging.StreamHandler()
        handler.setFormatter(JSONFormatter())
        logger.addHandler(handler)
        logger.setLevel(logging.INFO)
    return logger

```

PHASE 4: SOP MATCHER (Background Task)

4.1 app/tasks/sop_matcher.py — full file

This is the core async logic. It runs after every new enquiry is created. It does keyword matching, updates the DB, and auto-escalates if nothing matches.

```

python
from sqlalchemy.orm import Session
from app.models import Enquiry, Message, StatusTimeline
from app.logger import get_logger
from datetime import datetime

```

```

logger = get_logger("sop_matcher")

```

```

# — SOP Definitions

```

```

# Each SOP has a name, keywords to match against, and a suggested response.

```

```

SOPS = [

```



```

{
  "name": "Booking Enquiry",
  "keywords": ["book", "appointment", "schedule", "reserve", "slot", "availability"],
  "response": (
    "Thank you for reaching out! We'd love to book you in. "
    "Please let us know your preferred date and time and we'll confirm your appointment
shortly."
  ),
},
{
  "name": "Pricing Question",
  "keywords": ["price", "cost", "quote", "charge", "fee", "how much", "rate", "pricing"],
  "response": (
    "Great question! Our pricing depends on the specific service you're interested in. "
    "Could you share more details so we can give you an accurate quote?"
  ),
},
{
  "name": "Complaint",
  "keywords": ["complaint", "unhappy", "disappointed", "refund", "bad", "worst", "angry",
"upset", "wrong"],
  "response": (
    "We're really sorry to hear about your experience. "
    "A member of our team will review your concern and get back to you as soon as
possible."
  ),
},
{
  "name": "After-Hours Message",
  "keywords": ["after hours", "closed", "tomorrow", "weekend", "when do you open",
"office hours"],
  "response": (
    "Thanks for your message! Our office is currently closed. "
    "We'll get back to you during business hours (Mon–Fri, 9am–6pm). "
    "For urgent matters, please call our emergency line."
  ),
},
{
  "name": "General Follow-Up",
  "keywords": ["follow up", "following up", "any update", "heard back", "status", "checking
in"],
  "response": (
    "Thanks for following up! We're looking into your enquiry and will have an update for
you shortly."
  ),
},
]

```

```

def match_sop(message: str):
    """
    Returns the matching SOP dict or None.
    Lowercases the message and checks if any keyword is a substring.
    """
    message_lower = message.lower()
    for sop in SOPS:
        for keyword in sop["keywords"]:
            if keyword in message_lower:
                return sop
    return None


def process_enquiry(enquiry_id: str, db: Session):
    """
    Background task that:
    1. Fetches the enquiry
    2. Tries to match an SOP
    3. Updates the record with result
    4. Adds an AI message and a timeline event
    5. Auto-escalates if no SOP matched
    """
    logger.info("Background task started", extra={"enquiry_id": enquiry_id})

    enquiry = db.query(Enquiry).filter(Enquiry.id == enquiry_id).first()

    if not enquiry:
        logger.error("Enquiry not found in background task", extra={"enquiry_id": enquiry_id})
        return

    # Update status to processing
    enquiry.status = "processing"
    enquiry.updated_at = datetime.utcnow()
    _add_timeline(db, enquiry_id, "processing", "Background task started")
    db.commit()

    matched_sop = match_sop(enquiry.message)

    if matched_sop:
        # SOP matched — update with suggested response
        enquiry.sop_match = matched_sop["name"]
        enquiry.suggested_response = matched_sop["response"]
        enquiry.status = "sop_matched"
        enquiry.updated_at = datetime.utcnow()

        _add_message(db, enquiry_id, sender="ai", content=matched_sop["response"])
        _add_timeline(db, enquiry_id, "sop_matched", f"Matched SOP: {matched_sop['name']}")

```

```

        logger.info(
            "SOP matched",
            extra={"enquiry_id": enquiry_id, "sop": matched_sop["name"]}
        )

    else:
        # No SOP matched — auto-escalate
        enquiry.status = "escalated"
        enquiry.escalation_reason = "No SOP matched for inbound message. Flagged for
human review."
        enquiry.updated_at = datetime.utcnow()

        _add_timeline(db, enquiry_id, "escalated", "Auto-escalated: no SOP match found")

        logger.warning(
            "No SOP matched — auto-escalated",
            extra={"enquiry_id": enquiry_id}
        )

    db.commit()
    logger.info("Background task complete", extra={"enquiry_id": enquiry_id})

def _add_timeline(db: Session, enquiry_id: str, status: str, note: str):
    entry = StatusTimeline(
        enquiry_id=enquiry_id,
        status=status,
        note=note,
        timestamp=datetime.utcnow()
    )
    db.add(entry)

def _add_message(db: Session, enquiry_id: str, sender: str, content: str):
    msg = Message(
        enquiry_id=enquiry_id,
        sender=sender,
        content=content,
        timestamp=datetime.utcnow()
    )
    db.add(msg)

```

PHASE 5: ROUTE HANDLERS

5.1 app/routes/enquiry.py — full file

All 4 enquiry-related endpoints live here.

```
python
from fastapi import APIRouter, Depends, BackgroundTasks, HTTPException, status
from sqlalchemy.orm import Session
from datetime import datetime, timedelta
```

```
from app.database import get_db
from app.models import Enquiry, Message, StatusTimeline, FollowUp
from app.schemas import (
    EnquiryCreate, EnquiryCreatedResponse,
    FollowUpCreate, FollowUpResponse,
    EscalateCreate, EscalateResponse,
    EnquiryHistoryResponse
)
from app.tasks.sop_matcher import process_enquiry
from app.logger import get_logger
```

```
router = APIRouter(prefix="/enquiry", tags=["Enquiry"])
logger = get_logger("routes.enquiry")
```

— POST /enquiry

```
@router.post(
    "/",
    response_model=EnquiryCreatedResponse,
    status_code=status.HTTP_202_ACCEPTED,
    summary="Create a new inbound enquiry",
    description=(
        "Accepts a customer enquiry from WhatsApp, email, or call. "
        "Returns a job ID immediately and processes the enquiry asynchronously in the "
        "background."
    )
)
def create_enquiry(
    payload: EnquiryCreate,
    background_tasks: BackgroundTasks,
    db: Session = Depends(get_db)
):
    # Create the enquiry record
    enquiry = Enquiry(
        customer_name=payload.customer_name,
        channel=payload.channel.value,
        message=payload.message,
        status="new"
```

```

    )
    db.add(enquiry)

    # Store the original customer message
    customer_message = Message(
        enquiry_id=enquiry.id,
        sender="customer",
        content=payload.message,
        timestamp=datetime.utcnow()
    )
    db.add(customer_message)

    # Add initial timeline entry
    timeline_entry = StatusTimeline(
        enquiry_id=enquiry.id,
        status="new",
        note="Enquiry received",
        timestamp=datetime.utcnow()
    )
    db.add(timeline_entry)

    db.commit()
    db.refresh(enquiry)

    # Fire background task — non-blocking
    background_tasks.add_task(process_enquiry, enquiry.id, db)

    logger.info(
        "Enquiry created",
        extra={"enquiry_id": enquiry.id, "channel": payload.channel.value, "customer":
payload.customer_name}
    )

    return EnquiryCreatedResponse(job_id=enquiry.id)

```

— POST /enquiry/{id}/followup

```

@router.post(
   ("/{enquiry_id}/followup",
    response_model=FollowUpResponse,
    status_code=status.HTTP_201_CREATED,
    summary="Schedule a follow-up for an enquiry",
    description="Schedules a follow-up for an open enquiry after a given delay in minutes.
Optionally accepts a message template."
)
def schedule_followup(

```

```

    enquiry_id: str,
    payload: FollowUpCreate,
    db: Session = Depends(get_db)
):
    enquiry = db.query(Enquiry).filter(Enquiry.id == enquiry_id).first()
    if not enquiry:
        raise HTTPException(status_code=404, detail="Enquiry not found")

    if enquiry.status == "escalated":
        raise HTTPException(
            status_code=400,
            detail="Cannot schedule a follow-up on an escalated enquiry. Resolve the escalation
first."
        )

    scheduled_at = datetime.utcnow() + timedelta(minutes=payload.delay_minutes)

    followup = FollowUp(
        enquiry_id=enquiry_id,
        scheduled_at=scheduled_at,
        message_template=payload.message_template
    )
    db.add(followup)

    # Log it in the timeline
    timeline_entry = StatusTimeline(
        enquiry_id=enquiry_id,
        status=enquiry.status,
        note=f"Follow-up scheduled for {scheduled_at.isoformat()}",
        timestamp=datetime.utcnow()
    )
    db.add(timeline_entry)

    db.commit()
    db.refresh(followup)

    logger.info(
        "Follow-up scheduled",
        extra={"enquiry_id": enquiry_id, "scheduled_at": scheduled_at.isoformat()}
    )

    return followup

```

— POST /enquiry/{id}/escalate

@router.post(

```

"/{enquiry_id}/escalate",
response_model=EscalateResponse,
status_code=status.HTTP_200_OK,
summary="Escalate an enquiry to a human agent",
description="Marks an enquiry as escalated. Accepts a reason. Updates status and logs
the event in the timeline."
)
def escalate_enquiry(
    enquiry_id: str,
    payload: EscalateCreate,
    db: Session = Depends(get_db)
):
    enquiry = db.query(Enquiry).filter(Enquiry.id == enquiry_id).first()
    if not enquiry:
        raise HTTPException(status_code=404, detail="Enquiry not found")

    if enquiry.status == "escalated":
        raise HTTPException(status_code=400, detail="Enquiry is already escalated")

    enquiry.status = "escalated"
    enquiry.escalation_reason = payload.reason
    enquiry.updated_at = datetime.utcnow()

    timeline_entry = StatusTimeline(
        enquiry_id=enquiry_id,
        status="escalated",
        note=f"Manually escalated: {payload.reason}",
        timestamp=datetime.utcnow()
    )
    db.add(timeline_entry)

    db.commit()
    db.refresh(enquiry)

    logger.warning(
        "Enquiry escalated",
        extra={"enquiry_id": enquiry_id, "reason": payload.reason}
    )

    return EscalateResponse(
        id=enquiry.id,
        status=enquiry.status,
        escalation_reason=enquiry.escalation_reason
    )

```

— GET /enquiry/{id}/history

```

@router.get(
   ("/{enquiry_id}/history",
    response_model=EnquiryHistoryResponse,
    status_code=status.HTTP_200_OK,
    summary="Get full history of an enquiry",
    description="Returns the enquiry details, full message thread, and complete status
    timeline."
)
def get_history(
    enquiry_id: str,
    db: Session = Depends(get_db)
):
    enquiry = db.query(Enquiry).filter(Enquiry.id == enquiry_id).first()
    if not enquiry:
        raise HTTPException(status_code=404, detail="Enquiry not found")

    return enquiry

```

5.2 app/routes/health.py — full file

```

python
from fastapi import APIRouter, Depends
from sqlalchemy.orm import Session
from sqlalchemy import text
from app.database import get_db
from app.schemas import HealthResponse

router = APIRouter(tags=["Health"])

@router.get(
    "/health",
    response_model=HealthResponse,
    summary="Health check",
    description="Returns API status and database connectivity."
)
def health_check(db: Session = Depends(get_db)):
    try:
        db.execute(text("SELECT 1"))
        db_status = "connected"
    except Exception:
        db_status = "unavailable"

    return HealthResponse(status="ok", database=db_status)

```

PHASE 6: MAIN APP ENTRY POINT

6.1 app/main.py — full file


```
python
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from app.database import engine, Base
from app.routes import enquiry, health
from app.logger import get_logger

# Create all tables on startup
Base.metadata.create_all(bind=engine)

logger = get_logger("main")

app = FastAPI(
    title="Closira Enquiry API",
    description=(
        "Backend service that simulates Closira's core customer enquiry-handling workflow. "
        "Handles inbound enquiries, processes them asynchronously via SOP matching, "
        "and exposes status, escalation, follow-up, and history endpoints."
    ),
    version="1.0.0",
    contact={
        "name": "Closira Engineering",
    },
    license_info={
        "name": "Private",
    }
)

# CORS — open for now since there's no auth
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"],
)

# Register routers
app.include_router(enquiry.router)
app.include_router(health.router)

@app.on_event("startup")
async def startup_event():
    logger.info("Closira API starting up")

@app.on_event("shutdown")
async def shutdown_event():
    logger.info("Closira API shutting down")
```

PHASE 7: TESTS

7.1 tests/test_enquiry.py — full file

```
python
import pytest
from fastapi.testclient import TestClient
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker

from app.main import app
from app.database import Base, get_db

# Use a separate in-memory SQLite DB for tests
TEST_DATABASE_URL = "sqlite:///./test_closira.db"

engine = create_engine(TEST_DATABASE_URL, connect_args={"check_same_thread":
False})
TestingSessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

Base.metadata.create_all(bind=engine)

def override_get_db():
    db = TestingSessionLocal()
    try:
        yield db
    finally:
        db.close()

app.dependency_overrides[get_db] = override_get_db
client = TestClient(app)
```

— Helpers

```
def create_sample_enquiry(message="I'd like to book an appointment"):
    return client.post("/enquiry/", json={
        "customer_name": "Test User",
        "channel": "whatsapp",
        "message": message
    })
```

— Health

```
def test_health_check():
    res = client.get("/health")
    assert res.status_code == 200
    data = res.json()
    assert data["status"] == "ok"
    assert data["database"] == "connected"
```

— Create Enquiry

```
def test_create_enquiry_returns_job_id():
    res = create_sample_enquiry()
    assert res.status_code == 202
    data = res.json()
    assert "job_id" in data
    assert data["status"] == "processing"
```

```
def test_create_enquiry_missing_field():
    res = client.post("/enquiry/", json={
        "customer_name": "Test User",
        "channel": "whatsapp"
        # message missing
    })
    assert res.status_code == 422
```

```
def test_create_enquiry_invalid_channel():
    res = client.post("/enquiry/", json={
        "customer_name": "Test User",
        "channel": "telegram", # not a valid channel
        "message": "Hello"
    })
    assert res.status_code == 422
```

— History

```
def test_get_history_returns_data():
    res = create_sample_enquiry()
    job_id = res.json()["job_id"]
    history = client.get(f"/enquiry/{job_id}/history")
    assert history.status_code == 200
    data = history.json()
    assert data["id"] == job_id
```

```
assert data["customer_name"] == "Test User"
assert "messages" in data
assert "timeline" in data
```

```
def test_get_history_not_found():
    res = client.get("/enquiry/nonexistent-id/history")
    assert res.status_code == 404
```

— Follow-up

```
def test_schedule_followup():
    res = create_sample_enquiry()
    job_id = res.json()["job_id"]
    followup = client.post(f"/enquiry/{job_id}/followup", json={
        "delay_minutes": 30,
        "message_template": "Hi {name}, following up!"
    })
    assert followup.status_code == 201
    data = followup.json()
    assert data["enquiry_id"] == job_id
    assert data["done"] == False
```

```
def test_followup_not_found():
    res = client.post("/enquiry/bad-id/followup", json={"delay_minutes": 10})
    assert res.status_code == 404
```

— Escalation

```
def test_escalate_enquiry():
    res = create_sample_enquiry()
    job_id = res.json()["job_id"]
    escalate = client.post(f"/enquiry/{job_id}/escalate", json={
        "reason": "Customer is requesting a manager."
    })
    assert escalate.status_code == 200
    data = escalate.json()
    assert data["status"] == "escalated"
    assert "manager" in data["escalation_reason"]
```

```
def test_escalate_already_escalated():
    res = create_sample_enquiry()
    job_id = res.json()["job_id"]
```

```
client.post(f"/enquiry/{job_id}/escalate", json={"reason": "First escalation"})
second = client.post(f"/enquiry/{job_id}/escalate", json={"reason": "Second attempt"})
assert second.status_code == 400
```

```
def test_escalate_not_found():
    res = client.post("/enquiry/bad-id/escalate", json={"reason": "test"})
    assert res.status_code == 404
```

PHASE 8: RUNNING THE PROJECT

8.1 Start the server

```
bash
cd backend
source venv/bin/activate
uvicorn app.main:app --reload
```

API runs at: `http://localhost:8000` Swagger docs at:
`http://localhost:8000/docs` ReDoc at: `http://localhost:8000/redoc`

8.2 Run tests

```
bash
cd backend
pytest tests/ -v
```

PHASE 9: CURL TEST COMMANDS (for your submission)

Save these in a file called `api_tests.sh` in your repo root.

```
bash
# 1. Health check
curl http://localhost:8000/health

# 2. Create an enquiry (booking — will match SOP)
curl -X POST http://localhost:8000/enquiry/ \
  -H "Content-Type: application/json" \
  -d '{"customer_name": "Sarah M.", "channel": "whatsapp", "message": "Hi I would like to book an appointment for next Tuesday"}'

# Save the job_id from the response, e.g.:
# JOB_ID="paste-your-job-id-here"

# 3. Get history (replace JOB_ID)
curl http://localhost:8000/enquiry/$JOB_ID/history

# 4. Schedule a follow-up
```

```
curl -X POST http://localhost:8000/enquiry/$JOB_ID/followup \
-H "Content-Type: application/json" \
-d '{"delay_minutes": 30, "message_template": "Hi {name}, just following up on your booking request!"}'
```

5. Escalate

```
curl -X POST http://localhost:8000/enquiry/$JOB_ID/escalate \
-H "Content-Type: application/json" \
-d '{"reason": "Customer has called 3 times and is very frustrated. Needs immediate attention."}'
```

6. Create an enquiry that will auto-escalate (no SOP keyword)

```
curl -X POST http://localhost:8000/enquiry/ \
-H "Content-Type: application/json" \
-d '{"customer_name": "John D.", "channel": "email", "message": "Zxcvbnm asdfghjkl qwertyuiop"}'
```

PHASE 10: README CONTENT

Write your README.md covering all these points. This is what they'll read first.

Sections to include:

1. Setup & Run

- Prerequisites (Python 3.10+)
- Clone repo, create venv, install requirements, run uvicorn, open /docs

2. Database Schema & Reasoning

- Explain the 4 tables and their relationships
- Say: "SQLite was chosen for this prototype because it requires zero infrastructure setup, ships inside the repo, and is sufficient for a single-process evaluation environment. In production, this would be replaced with PostgreSQL to support concurrent writes, connection pooling, and multi-tenant isolation."

3. BackgroundTasks vs Celery Decision

- Say: "FastAPI BackgroundTasks was chosen over Celery because this prototype runs in a single process and doesn't require distributed task queuing, persistent retry logic, or a message broker like Redis. Celery would be the right choice when tasks need to survive server restarts, be monitored via Flower, or be distributed across multiple workers. For this scope, BackgroundTasks keeps the setup simple and the trade-off is explicitly acknowledged."

4. API Endpoints Summary — table with method, path, description

5. SOP Matching Logic — list the 5 SOPs and their keywords

6. Known Limitations / Trade-offs

- Background task shares the same DB session as the request — in production you'd use a separate session factory
- No authentication or tenant isolation (would add API key middleware in production)
- No actual scheduling engine for follow-ups (stored in DB but not dispatched)
- SQLite has no concurrent write support

7. Running Tests

PHASE 11: COMMIT HISTORY STRATEGY

They explicitly look at commit history. Make clean, meaningful commits as you go:

```
git commit -m "init: project structure and requirements"
git commit -m "feat: database models and SQLAlchemy setup"
git commit -m "feat: pydantic schemas for all endpoints"
git commit -m "feat: JSON structured logger"
git commit -m "feat: SOP matcher background task"
git commit -m "feat: enquiry routes (create, followup, escalate, history)"
git commit -m "feat: health check endpoint"
git commit -m "feat: FastAPI app entry point with CORS and startup events"
git commit -m "test: full test suite for all endpoints"
git commit -m "docs: README with setup, schema, and decision rationale"
git commit -m "chore: curl test commands and api_tests.sh"
```

QUICK REFERENCE — EVALUATION CHECKLIST

Before you submit, verify every item:

- All 5 endpoints working and returning correct status codes
- /docs fully populated with descriptions and example payloads
- Background task fires on POST /enquiry and updates status
- SOP matching works for booking, pricing, complaint, after-hours, follow-up keywords
- Auto-escalation triggers when no keyword matches
- JSON structured logs printing to console for every key event
- 400/404/422 errors return meaningful messages, never crash
- All tests passing with `pytest -v`
- README covers setup, schema, BackgroundTasks reasoning, trade-offs
- Clean commit history (10+ meaningful commits)
- Video walkthrough: show /docs, hit each endpoint, explain SOP task, show logs